P2964R1
Adding support for user-defined element types (UDT) in `std::simd`

Published Proposal, 2024-05-22

This version:
http://wg21.link/P2964R1

Authors:
Daniel Towner (Intel)
Ruslan Arutyunyan (Intel)

Audience:
LEWG

Project:
ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Abstract

The paper proposes to support user-defined element types in `std::simd` using customization points

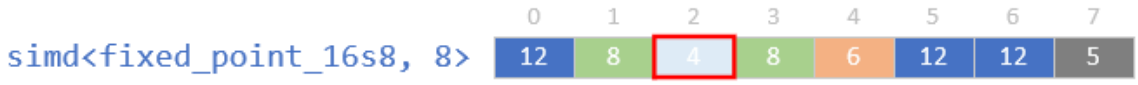
Table of Contents

§ 1. Motivation

ISO/IEC 19570:2018 (1) introduced data-parallel types to the C++ Extensions for Parallelism TS. [P1928R8] is proposing to make that a part of C++ IS. In the current proposal individual elements must be of a type which satisfies constraints set out by `std::simd` (e.g., arithmetic types, complex-value types). In this document we describe how `std::simd` could allow user-defined element types too, with automatic generation of element-wise operators for user-defined types which define those operators, and with the option of dispatch of operations to customization functions which allow the user to provide more efficient implementations when automatic generation does less well.

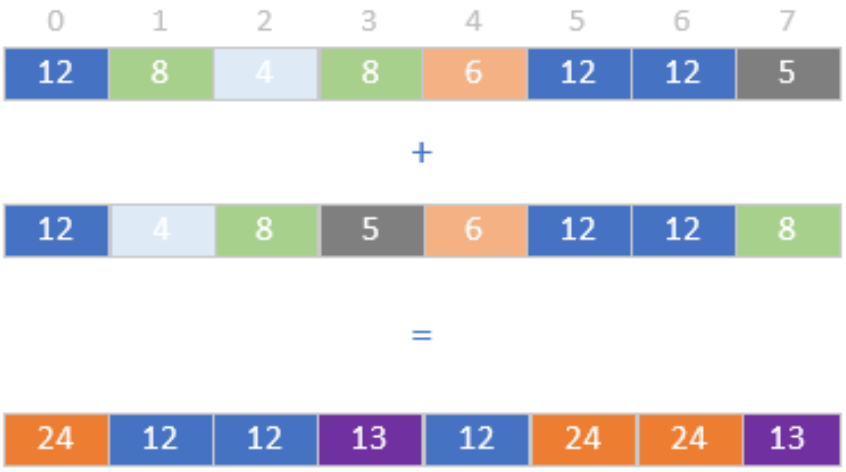
Being able to have user-defined types in a `std::simd` value is desirable because it allows us to build on top of the standard features of `std::simd` to support SIMD programming in specialised problem domains, such as signal or media processing, which might have custom data types (e.g., saturating or fixed-point types). The idea is that `std::simd` will be used to provide generic SIMD capabilities for performing loads, stores, masking, reductions, gathers, scatters, and much more, and then a set of customization points are provided to allow the fundamental arithmetic operators to be overloaded where needed.

As a concrete example, consider that we might have a fixed-point data type for signal processing called `fixed_point_16s8`. Such a fixed-point data type allows a fractional (non-integer) value to be stored with only a fixed number of digits to represent their fractional part. An instruction set which targets digital signal processing will be likely to have instructions which accelerate the fundamental arithmetic operations for these types and our aim is to allow `simd` values of this underlying fixed-point element type to be created and used. To begin with, a `simd<fixed_point_16s8>` is represented or stored using individual blocks of bits representing each element of the user-defined type within a vector register:



Note that a selected element, highlighted with a red rectangle, represents an individual `fixed_point_16s8` element value. The colours and values in each box represent specific bit patterns. Where two elements have the same bit-pattern/colour, they represent the same element value. An operation which moves elements within or across `simd` objects, or to and from memory, will copy the bit patterns. The elements don't change value because they move. In the diagram below a `permute` has been used to extract the even elements of our original example above.

Note that an element representing a specific value in this example (e.g., the dark-blue value 12) continues to represent the value 12 regardless of where it appears within the `simd` object. However, there are also cases where the bit pattern of individual elements does need to be interpreted as a specific value. For example, if we wish to add two `simd<fixed_point_16s8>` values together then we need to define how to efficiently handle the addition of specific bit patterns to form a new bit pattern. In the example below we are adding two `simd<fixed_point_16s8>` values and expecting to get a result which is created by calling some underlying target-specific function:



In this example the `std::simd` objects themselves have no knowledge of this since the elements are of a custom type, so we need to encode that knowledge into a user-defined customisation point. There will be a customisation point in `std::simd` in those places where knowledge of what the bit-pattern in a user-defined `simd` element actually means. Common examples of such points are arithmetic operations or relational comparators. In the example above, a customisation point for `simd::operator+` will be provided to map onto a suitable hardware instruction to perform a vector fixed-point addition.

Custom types will not always support every possible operator that is exposed by `std::simd`. For example, perhaps a `fixed_point_16s8` doesn't allow division or modulus, in which case `std::simd::operator/` and `std::simd::operator%` must be removed from the overload set entirely.

Putting these mechanisms for defining custom element types together allows us to write generic functions which can then be invoked with both built-in and user-defined types equally easily, using the rich API of `std::simd`, even when the user-defined types are using target-specific customised functions:

```
// Compute the dot-product of a simd value.
template<typename T, typename ABI>
auto dotprod(const basic_simd<T, ABI>& lhs, const basic_simd<T, ABI>& rhs) {
  // The reduction moves are handled bit-wise, while the multiply and addition are delegate
  // to target-specific customisation points. The high-level code is unaltered.
  return reduce(lhs * rhs, std::plus<>{});
}

...
float dfp = dotprod(simdFloat0, simdFloat1);
fixed_point_16s8 dfxp = dotprod(simdFixed0, simdFixed1);
```

An extended example of some custom element types is given toward the end of this paper, along with our experiences with using them.

This proposed extension of `std::simd` to support custom types applies only to types which can be vectorised as atomic units. That is, a `simd<T>` for a custom type `T` has storage which is the same as an array of those types (i.e., this might be called an array-of-structures). This is in contrast to another way of representing a parallel collection of custom element types in which the structure of the custom element is broken down into individual pieces (i.e., a structure-of-arrays). For example, given `simd<std::pair<A, B>>` the structure-of-array style of customisation would treat it as though it were `std::pair<simd<A>, simd<B>>`. While that could be a valuable feature for future consideration, it is different to what is being proposed in this paper.

In addition to allowing customized element types in `std::simd`, a side-effect of this paper is to set out a way of thinking about the meaning of different `std::simd` operations which makes the division of responsibility of different `std::simd` APIs clear. This makes it easier to discuss the related topic of adding new C++ builtin element types such as `std::byte`, and scoped/unscoped enumerations. For example, as we shall describe later there are specific basis functions which define the fundamental arithmetic behaviour of `std::simd` types, and knowing what those basis functions should be makes it easy to reason about how to support `enum` and `std::byte` types. Later in this document we shall introduce the necessary changes to allow `simd<enum>` and `simd<byte>` to be easily incorporated.

Our intent with user-defined types is that the underlying type behaves like a value type as defined in Elements of Programming (e.g., integers represented as two's complement values, or rational numbers represented as a concatenation of two 32-bit sequences, interpreted as integer numerator and denominator). Only arithmetic operations can be applied to value-types. `std::simd` does not provide overloads for non-value-type operators, such as the member-related operators like `operator->`, `operator,`. We do not intend to extend `std::simd` to include these non-value operators.

Since `std::simd` is built on top of native hardware support, there are certain limitations in the hardware that prevent arbitrary types being representable in a `std::simd`. It will be necessary to impose some restrictions on the types of elements to make them implementable.

Note that a user-defined type, such as our example `fixed_point_16s8`, might provide its own overloaded operators for handling the different types of arithmetic operation that could occur. Ideally we want the operators from that scalar type to be mapped over to work for `simd<fixed_point_16s8>` as well. In effect, every `simd` operator for that type would perform the equivalent element-wise operator on the underlying values. For example, suppose we have the following partially-implemented scalar `fixed_point_16s8` type:

```
struct fixed_point_16s8 {

    fixed_point_16s8 operator+(fixed_point_16s8 lhs, fixed_point_16s8 rhs)
      { return __intrin_fixed_add(lhs, rhs); }
    fixed_point_16s8 operator-(fixed_point_16s8 lhs, fixed_point_16s8 rhs)
      { return __intrin_fixed_sub(lhs, rhs); }

    int do_special_op() const;

    std::int16_t data;
};
```

If the user instantiated `simd<fixed_point_16s8>` and performed addition between two values of this parallel type then this should give the same result as invoking the operator above on each element in turn. The automatic extension of a scalar operator to a simd operator is the first level of support that `simd<>` should provide. Unfortunately, there may be cases

where the compiler does a poor job of auto-vectorising such code; iterating over each element in turn does not automatically make good simd code. For these cases we also need a mechanism which allows the automatic operator inferencing to be overridden to replace it with a more efficient method provided by the user. We will use the idea of customisation points, as used in other parts of C++, to enable this. The user will be able to provide a special function for specific simd operators to provide efficient implementations for those cases where the compiler doesn't find this for itself.

In the example above we also had a class method. At the moment no mechanism exists in C++ to allow us to automatically add the equivalent method to `simd<fixed_point_16s8>` so we cannot create a complex simd-generic extension of the user-defined type yet. In future if suitable reflection capabilities are added this extension may become possible, but this is currently out of scope for this paper.

§ 2. Understanding customization opportunities, requirements, and restrictions

The first step in understanding how `std::simd` can be extended to new element types is to review what operations are provided by `std::simd`, and how those operations must adapt to operating on custom user-defined elements.

§ 2.1. Type restrictions

The current proposal for `std::simd` in [P1928R8] only allows selected types to be stored in a `std::simd` but our proposal for user-defined types makes it possible to theoretically store any other C++ type. It may not make sense to be able to store some of those other types in a `simd`. For example, they may have side-effects that means they don't work as expected when operated on in parallel, or they rely on features that don't translate well to a SIMD instruction set. We should seek to restrict the valid element types to avoid the worst issues that might arise.

Although [P1928R8] makes no mention of it, there is an implication that the elements in a `std::simd` are trivially copyable. For operations which move elements around - broadcast, permutation, gather, scatter, and so on - it is assumed that when the bits move location within a `std::simd` object they will continue to represent their original value. `std::simd` currently restricts types to be floating-point, integral, or complex-valued (with [P2663R4]), all of which are trivially copyable.

Ultimately the success of `std::simd` is tied to how well the underlying hardware can support the element type. All known hardware supports only elements which have sizes which are power-of-2, so we propose to require `simd` element types to respect this. Also, most hardware has a limit on the size of individual elements. In the current proposal of `std::simd`, the largest possible element type is the 128-bit `std::complex<double>`, so we will set this as the upper limit of user-defined elements. Therefore, any user-defined type for `simd` must be 1, 2, 4, 8 or 16 bytes in size.

`simd` was designed to work on arithmetic value-like types, in common with the hardware instruction sets to which they are ultimately giving access. Restricting `simd` to only such types is difficult as C++ has no way of querying a type to determine if it is a product or sum type. Instead, we will only extend user-defined operators to those operators which `std::simd` already supports (e.g., `operator+`). We will not provide additional operators to support member-like access (e.g., `operator->*`), dereferencing (e.g., unary `operator*`) since the presence of these operators implies they aren't suitable for storing in a `std::simd` anyway

We propose that certain types should always be banned as `simd` elements including `simd<union>`, and any sort of pointer element. In the proposal we will also explore the use of an opt-out mechanism which can be used to prevent certain types from ever being contained in a `simd`.

§ 2.2. Customisation point classifications

While many operations within `std::simd` will work on trivially copyable custom types there are also places where `std::simd` does need to interpret the meaning of the bits, and it is those that need to be customization points. Each function can be put into one of the following categories:

Basis
: A basis function is one that must be provided as a customization point to allow the underlying element type to be used. An example would be addition; if addition is not provided as a customization point for a user-defined type then `std::simd` values of that type cannot be added together.

Custom
: A customization function is one that can be implemented generically but which can also be customized to provide a more efficient implementation if one exists. An example of a Custom function would be negate (`operator-`) which could have a default implementation which subtracts from zero, or could be customized if the type provides a faster alternative (e.g., sign-bit flip for a floating-point-like type).

Copy
: A copy function uses the trivially copyable nature of the underlying type and allows bits to be moved from one place to another. The `permute` function is a good example of a Copy function since a `std::simd` of any type can move its elements around within a SIMD value without needing to know what the bits represent.

Algorithm
: An algorithm function uses other functions to implement some feature. If the algorithm relies on a Basis or Custom function which is not provided by the user-defined type then the algorithm function is removed from the overload set. An algorithm function does not provide a customization point.

The following table lists the key functions in `std::simd`, and what category they fall into. The table allows us to reason about what functions in `std::simd` will just work as they are currently defined, and to separate out those functions which must have customisation points defined in order to allow their behaviour to be changed for custom user-defined types.

Function	Type	Notes
`simd_mask`		
`simd_mask {}`	n/a	Virtually every function in `simd_mask` will work on user-defined types, with the exception of those listed in the next row. The mask is only dependent on knowing the number of bits in the type, and not on the interpretation of those bits.

`simd_mask::operator+`
`simd_mask::operator-`
`simd_mask::operator~`
	Algorithm	Convert and broadcast a 0, 1 or -1, and perform a `simd_select` using the mask to choose the appropriate value. Only provided if the convert is available.

Constructors		
`basic_simd(U)`	Copy	Broadcast a copy of the bits from represent the scalar source object to every element
`basic_simd(basic_simd<U>)`	Copy/custom	When U is the same as T, direct copy the bits into place. When U is different, we need a customization point to convert the user-defined elements. If no customization point is available, no conversions will be allowed but copying from other `std::simd` of the same type will be permitted.

Function	Type	Notes
`basic_simd(Gen&&)`	Copy	Each invocation of the generator builds an individual scalar value of the element type which is bitwise copied into the respective element.
`basic_simd(Iter)`	Copy/Algorithm	When `Iter::value_type` is the same, copy the bits. When `Iter::value_type` is different and a customization point exists for the conversion, create the `std::simd` as the value iterator type first, and then copy the bits. No customization point will be allowed since it is unlikely that it brings any performance benefit, although this decision can be revisited.
Copy functions		
`copy_to`	Copy/Algorithm	When the destination type is the same use a direct bit copy from the `simd` into memory. When the destination type is different, convert to a `std::simd` of the destination type and invoke `copy_to` on that. A customization point is not provided since it is highly unlikely that any hardware support is available for copying to an special type. If the destination type is different and there is no conversion customization point remove the conversion-copy from the overload set.
`copy_from`	Algorithm	Equivalent to calling `basic_simd(Iter)` and performing an assignment.
Subscript operators		
`operator[]`	Copy	Bitwise copy from element into the scalar output value
Unary operators		
`operator-`	Custom	If a customization point or builtin-type support is available use that. Otherwise if `operator-` is available use `simd<T>() - *this`. Otherwise remove from the overload set.
`operator--/++`	Algorithm	If `operator+/-` are available use `*this +/- T(1)`. Otherwise remove from the overload set.
`operator!`	Custom	If a customization point or builtin-type support is available use that. Otherwise if `operator==` is available return `*this == simd<T>()`. Otherwise remove from the overload set.
`operator~`	Basis	If a customization point or builtin-type support is available use that. Otherwise remove from the overload set.
Binary operators		
`operator+,-,*,/,%, <<,>>,&,|,^`	Basis	If a customization point or builtin-type support is available use that. Otherwise remove the `simd::operatorX` from the overload set.
Compound assignment operators		
`operator+=,-=,*=,/=,%=&=,|=,^=,<<=, >>=`	Algorithm	If a customization point or builtin-type support is available for the underlying operation use `*this = *this OP rhs`.

Function	Type	Notes
		Otherwise remove this from the overload set.
Relational operators		
`operator==`	Basis	If a customization point or builtin-type support is available call that. Otherwise remove this from the overload set. Note that although each element represents its values using a specific copyable bit pattern this doesn't mean that the same bit pattern represents an equal value (e.g., floating point NaN bit patterns will never be equal).
`operator!=`	Custom	If a customization point or builtin-type support is available call that. Otherwise if `operator==` is provided, return the negation of that function. Otherwise remove from the overload set.
`operator<,<=,>, >=`	Basis	If a customization point or builtin-type support is available call that. Otherwise remove from the overload set. Note that a minimal set could be provided (e.g., `operator<` and `operator==` since everything else can be built from those).
Conditional operator		
`simd_select`	Copy	Conditionally copy element bits with no interpretation.
Permute		
`permute`/permute-like	Copy	All permutes (generated or dynamic) move bits from one location to another without interpretation. Related operations like resize, insert, extract work in the same way.
`compress`/`expand`	Copy	All compression and expansion operations move bits from one location to another without interpretation.
`gather_from`	Copy	Gather the values as though they were a `simd<custom-storage-type>` and then use `std::bit_cast` to convert (at no cost) into a `std::simd` of the user defined type.
`scatter_to`	Algorithm	If the same type, bitwise scatter individual elements to the range using direct bitwise copy. If the destination type is different construct a `std::simd` of the destination type and perform a scatter on that type instead.
Reductions		
`reduce`	Algorithm	All reduction operations can be implemented using a sequence of permutes and arithmetic operations. If the desired operation for the reduction step is not available in the overload set then the corresponding reduction is also removed from the overload set. No customization point will be provided for reductions since it is unlikely that custom types will have hardware support for reductions. These can be added later if this is found to be untrue.

Function	Type	Notes
Free functions		

`min`
`max`
`clamp`
	Custom	If the user provides their own ADL overloaded customization point for this function then that will be used.
		Otherwise if relational operators are available for the type, use those to synthesise this operation (i.e., `simd_select(a < b, a, b)` for `min`).
		Otherwise remove from the overload set.

`abs`
`sin`
`log`
etc.
	Custom	For any other free functions an ADL overload can be provided by the user to handle that specific type.

§ 2.3. Required customization points

In the table of function classifications above we can discount the Copy functions from any further thought in this proposal. By limiting `std::simd` elements to those which are trivially copyable, we can provide any sort of operation which moves bits around using the `std::simd` implementation itself, with no special consideration for user-defined types.

Unsurprisingly, the table above shows us that we need customization points for all numeric operations, including:

```
plus          minus         negate          multiplies
divides       modulus

bit AND       bit OR        bit NOT         bit XORr

equal to      not equal to
greater       less          less or equal   greater or equal

logical AND   logical OR    logical NOT

shift_left    shift_right
```

Also unsurprisingly, these names are all those of the C++ transparent template wrappers, with the exception of shift-left and shift-right. The only other customisation point that would be needed is a conversion function. If a UDT can be constructed or copied from a different type `T`, then it should also be possible to construct or copy a `simd<UDT>` from `simd<T>` by element-wise application.

> NOTE: As a small aside, it isn't clear why transparent operators are not provided for shift operations, and perhaps they should be added in for completeness in the future.

For each of these customisation points we have three layers of behaviour:

- Default operator for standard C++ types (e.g., `int`, `float`, `complex<floating_point>`). No customisation is possible for any of the types available in C++.
- Explicit simd customisation. The user provides a specific function which is used to perform that operation on a `simd<UDT>`. The intent is to allow the user to make the operator as efficient as possible for cases where the compiler

may not auto-vectorise efficiently.

- Implicit `simd` customisation using the scalar type's own operator, applied element-wise. Ideally the compiler will auto-vectorise this to generate efficient code.
- Otherwise the operator is removed from the overload set.

For this last point, an example would be a user-defined complex type which might provide addition, subtraction and multiplication, but remove modulus, relational operators, and bitwise operators from the overload set, along with any other operations which depend upon those (e.g., compound modulus, compound bitwise).

Conversions will work in this customisation framework in the same manner as arithmetic operators; if a conversion is not explicitly defined, or the scalar type doesn't support it, then the `simd` type will also not support that conversion.

§ 3. Creating a customization framework

There are several considerations in a framework for user-defined element types: opt-in/out, storage, unary/binary operators and conversions, and free-functions. In this section we shall look at each of these in turn.

§ 3.1. Opt-in or opt-out

When a `simd<UDT>` is created the aim is for library to do as much work as possible to make that type behave reasonably, subject to a few restrictions (e.g., element size). All bit-copy operations will work on that type, and any operators defined for the scalar user-defined type will be mapped into the simd space.

In the original draft of this proposal the argument was made for an opt-in process, whereby the user would have to explicitly arrange for the user-defined type to be permitted as an element of a `simd`. Since that first proposal we have refined the behaviour of simd to allow it to infer simd operators from scalar operators, thereby making it possible to create a correctly behaving `simd<UDT>` with very little effort. With this new approach it seems unnecessarily onerous to have to require the user to opt-in to something that works on its own.

Our new proposal is to have an opt-out mechanism, where the user can explicitly indicate that a specified type is not suited to being an element in a `std::simd`, even if the type is otherwise legal (e.g., copyable, power-of-2, smaller than 16 bytes). Such an opt-out mechanism can be used to disable unsuitable user-defined types as well as other non-vectorisable types in the standard library, such as `simd<std::atomic>`.

§ 3.2. Storage

In order to perform Copy-like operations on `std::simd` elements we need to be able to inform `std::simd` values of how to store and move the underlying elements. In [P2964R0] we allowed the user to specify what the underlying storage should be, but with further thought we have decided to make the storage unspecified. The `simd` implementation is free to choose any storage type. Customisation points which use that storage will the use `std::bit_cast` to convert to and from that storage and the user-defined type as appropriate.

§ 3.3. Unary and binary operator customization points

There are many different ways to implement customization points, including template specialization, CPO, or `tag_invoke`. Which mechanism is most suitable can be discussed further if necessary but for this paper proposal we only care about whether customization should be allowed, not the exact mechanism that will be used.

All of the operators for `std::simd` are `friend` functions in order to allow ADL. We must leave these `friend` function operators, but allow them to defer to a customization point as required. One possible pseudo-implementation of an individual operator may be this:

```
constexpr friend basic_simd operator+(const basic_simd& lhs, const basic_simd& rhs)
requires (details::simd_has_custom_binary_plus || details::element_has_plus)
{
    if constexpr (details::is_standard_simd_type)        // int, float, complex, etc.
        return details::plus(lhs, rhs);
    else if constexpr (details::simd_has_custom_binary_plus)  // user customisation point
        return simd_binary_plus(lhs, rhs);
    else
        return details::element_wise_plus(lhs, rhs);     // Infer from scalar operator
}
```

In this example `operator+` is only put in the overload set if a builtin-arithmetic type supports addition directly or has a suitable customisation point. Internally, the function will then invoke the appropriate implementation.

We need to specify what the customisation points will be called to allow them to be discoverable. In the example above we have explicitly named the customisation function `simd_binary_plus`. This has the advantage that it is very clear and unambiguous in what it does, but it does introduce potential for high levels of duplication. This is because every operator will have its own unique customisation point name.

An alternative is to exploit the transparent templates that already exist in C++ and use them to differentiate between operations. Here is an example of the signature of a customisation point for a user defined type:

```
template<typename Abi, typename CustomType, std::invocable<CustomType, CustomType> Fn>
constexpr auto
simd_binary_op(const basic_simd<CustomType, Abi>& lhs,
               const basic_simd<CustomType, Abi>& rhs,
               Fn op);
```

This has a unique and distinctive name to mark it as a customisation point, and as a binary operator it takes in two `simd<custom-type>` inputs. It also takes a third parameter which specifies what binary operation to perform, chosen from the list of standard template wrappers. For example, the call site in `operator+` would look like this:

```
return simd_binary_op(lhs, rhs, std::plus<>{});
```

Similarly, unary operators can be customized using a customization function called `simd_unary_op` which accept a unary transparent template wrapper.

The advantage of using this mechanism rather than named functions for every required operator is that it removes the need for many different functions, and allows related operations to be consolidated into a single function. It also allows the transparent operator itself to be invoked directly to perform an operation. For example, suppose we want to define a customisa-

tion point for a user defined type that has non-standard behaviour for multiply and divide, but everything else works like a standard arithmetic operator (examples of such types include complex numbers and fixed-point numbers). The following pseudo-implementation captures this behaviour:

```
template<typename Abi, typename CustomType, std::invocable<CustomType, CustomType> Fn>
constexpr auto
simd_binary_op(const basic_simd<CustomType, Abi>& lhs,
               const basic_simd<CustomType, Abi>& rhs,
               Fn op)
{
  // Special case for some operators
  if      constexpr (std::is_same_v<Fn, std::multiplies<>>) return doCustomMultiply(lhs, rh
  else if constexpr (std::is_same_v<Fn, std::divides<>>)    return doCustomDivides(lhs, rhs
  // All other cases defer to an integer instead.
  else return op(simd<int>(lhs), simd<int>(rhs));
}
```

This is not only less verbose but it also makes it obvious how and why the custom type has to be handled differently to a builtin-type.

Unfortunately shift operators don't have transparent wrappers, so if we did use this approach we need one of the following too:

- a specially named customization point (e.g., `simd_shift_left_op`)
- an additional transparent operator added to `std::simd` to allow the existing binary operation to be used (e.g., `std::simd_shift_left<>`)
- a standardised `shift_left` transparent operator (i.e., `std::shift_left<>`)

In the Intel example implementation we have used the second of these. Having a different name and mechanism for shifts introduces extra complexity and non-uniformity. We hope that a transparent operator wrapper for shift might be added in future, in which case it will also be easier to transition to using that if we provide a local alternative to begin with.

§ 3.4. Conversion customization

Conversions behave much like the customization points for arithmetic operators. Like the other customisation operators conversions try three different strategies:

- If a customisation for `simd<UDT>` conversion exists, use that
- Otherwise if scalar conversion exists, invoke that element-wise
- Otherwise remove conversion capabilities

These conversion rules will be used wherever needed with `std::simd`, not just within the main constructor. For example, `copy_from` will load data from memory into a `simd` and then convert it to the desired output type.

§ 3.5. Overloads for free-functions

Anything outside `std::simd` itself can be freely overloaded for the custom type. For example, `abs` could be provided as follows:

```
template<typename Abi>
constexpr auto abs(const basic_simd<fixed_point_16s8, Abi>& v) {
    return /* special-abs-impl */;
}
```

No `std::simd`-specific customization points are required for any of the other functions as overloads will suffice.

§ 4. Extending support to `enum` and `std::byte`

It is useful to be able to create a `simd` of enumerations and `std::byte` and the mechanisms defined in this proposal make this trivial to implement. However, we would also define the following free-functions to mirror the support available with these standard types.

```
template<class Enum, typename Abi>
constexpr std::basic_simd<std::underlying_type_t<Enum>, Abi>
  to_underlying(std::basic_simd<Enum, Abi> se) noexcept;
```

```
template <class IntegerType, typename Abi>
constexpr std::basic_simd<IntegerType, Abi>
  to_integer( std::basic_simd<std::byte, Abi> b ) noexcept;
```

§ 5. Implementation Experience

In Intel's implementation of `std::simd`, the customization points described in this proposal have been implemented so that we can use instantiations of `std::simd` which use signal processing data types such as fixed-point or saturating integrals. In the following example we show code from our implementation which has been used to create a `simd` of a user defined saturating data type. We start by showing how the compiler does a reasonable job of inferring the required operators, but that certain operators prove too difficult to do well, at which point we can provide customisation points to smooth things out. We expect that a similar process will be used when other user-defined types are implemented.

Consider what might be needed to make a saturating data type. To begin with, we might already have a 16-bit scalar saturating data type:

```
struct saturating_int16
{
    saturating_int16(int v) : data(v) {}
    int16_t data;

    // Addition
    friend saturating_int16 operator+(saturating_int16 lhs,
                                      saturating_int16 rhs) {
```

```
        auto r = int32_t(lhs.data) + int32_t(rhs.data);
        return int16_t(std::min<int32_t>(std::max<int32_t>(r, -32768), 32767));
    }

    friend bool operator>(saturating_int16 lhs,
                          saturating_int16 rhs) {
      return lhs.data > rhs.data;
    }

    // Other operators also defined, but omitted for brevity.
};
```

Let us begin with simple permute operations which illustrates that storage and bit-copy operations will work as expected:

C++ Code	Assembly

`auto broadcast(int16_t x)`
`{`
`  return simd<saturating_int16>(x);`
`}`
	`broadcast(short):`
	`  vpbroadcastw  zmm0, edi`
	`  ret`

`auto iq_swap(const simd<saturating_int16>& v)`
`{`
`  return permute(v, [](auto idx) {`
`  return idx ^ 1;`
`  });`
`}`
	`iq_swap([...]> const&): #`
	`  vprold  zmm0, zmmword ptr [rdi], 16`
	`  ret`

Our original type defined addition, so we should be able to automatically call addition and its derivatives for the corresponding user-defined type:

C++ Code	Assembly

`auto add(simd<saturating_int16> lhs,`
`  simd<saturating_int16> rhs)`
`{`
`  return lhs + rhs;`
`}`
	`add([...]): #`
	`  vpaddsw ymm0, ymm0, ymm1`
	`  ret`

`auto compound_add(simd<saturating_int16> lhs,`
`  simd<saturating_int16> rhs)`
`{`
`  lhs += rhs;`
`  return lhs;`
`}`
	`compound_add([...]): #`
	`  vpaddsw ymm0, ymm0, ymm1`
	`  ret`

Note that the compiler (Intel oneAPI 2024.0) has done an excellent job, even generating a real saturation instruction. Unfortunately there is a small issue with the quality of the generated code, which we shall return to shortly.

Next, we need to be able to compare saturated values. Our original class was able to do this using its own operator, allowing us to unlock comparisons including reduction comparisons.

C++ Code	Assembly

`auto cmp_lt(simd<saturating_int16> lhs,`
`  simd<saturating_int16> rhs)`
`{`
`  return lhs > rhs;`
`}`
	`cmp_lt([...]): #`
	`  vpcmpgtw  ymm0, ymm1, ymm0`
	`  ret`

`auto biggest(simd<saturating_int16> lhs,`
`  simd<saturating_int16> rhs)`
`{`
`  return max(lhs, rhs);`
`}`
	`biggest([...]): #`
	`  vpmaxsw ymm0, ymm0, ymm1`
	`  ret`

Although the compiler has done a good job of the examples above, there were unfortunately a few places where it didn't do so well. For example:

C++ Code	Assembly

`auto reduce_add(simd<saturating_int16> v)`
`{`
`  return reduce(v, std::plus<>{});`
`}`
	`reduce_add([...]):`
	`  vextracti128  xmm1, ymm0, 1`
	`  vpaddsw xmm0, xmm0, xmm1`
	`  vpextrq rdx, xmm0, 1`
	`  vmovq  rax, xmm0`
	`  mov  rsi, rax`
	`  shr  rsi, 48`
	`  mov  rcx, rdx`
	`  shr  rcx, 48`
	`  lea  edi, [rsi + rcx]`
	`  movsx  edi, di`
	`  sar  edi, 15`
	`  xor  edi, -32768`
	`  add  si, cx`
	`  cmovo  esi, edi`
	`  ...`

For an unknown reason the compiler has switched to using element-by-element application of the scalar operation here, which is considerably slower. It is likely that we will fix the compiler to correct whatever mistake it is making here, but for now we can use the customisation point mechanism to aid the compiler in doing a better job. We want to change the behavior of `std::simd` so that addition is handled explicitly using the known intrinsics. We do this by defining our customisa-

tion function. Here is a very simple implementation which runs on an Intel AVX2 machine. It could be extended to work on all Intel instruction sets (e.g, AVX-512) but that is outside the scope of this proposal.

```
template<typename Abi>
constexpr auto simd_binary_op(const xvec::basic_simd<saturating_int16, Abi>& lhs,
                              const xvec::basic_simd<saturating_int16, Abi>& rhs,
                              std::plus<>)
{
    constexpr int numElements = xvec::basic_simd<saturating_int16, Abi>::size;
    if constexpr (numElements <= 8)
        return basic_simd<saturating_int16, Abi>(
                 _mm_adds_epi16(lhs.to_register(), rhs.to_register()));
    else
        return basic_simd<saturating_int16, Abi>(
                _mm256_adds_epi16(lhs.to_register(), rhs.to_register()));
}
```

This now generates code like this:

C++ Code	Assembly

`auto reduce_add(simd<saturating_int16> v)`
`{`
`  return reduce(v, std::plus<>{});`
`}`
	`reduce_add([...]):`
	`  vextracti128  xmm1, ymm0, 1`
	`  vpaddsw xmm0, xmm0, xmm1`
	`  vpshufd xmm1, xmm0, 238`
	`  vpaddsw xmm0, xmm0, xmm1`
	`  vmovq  rax, xmm0`
	`  vmovd  xmm0, eax`
	`  shr  rax, 32`
	`  vmovd  xmm1, eax`
	`  vpaddsw xmm0, xmm0, xmm1`
	`  vmovd  eax, xmm0`
	`  shr  eax, 16`
	`  vmovd  xmm1, eax`
	`  vpaddsw xmm0, xmm0, xmm1`
	`  vmovd  eax, xmm0`
	`  vzeroupper`
	`  ret`

This is acceptably better code.

§ 5.1. Summary of implementation experience

In this section we have given a simple example in which we started with a scalar user-defined type and automatically inferred the `simd` versions of its operators, allowing us to quickly develop code which used `simd<UDT>`. During inspection of the generated assembly code we found an issue with the quality of the code which we were able to overcome by using a customisation point to help guide the compiler to generate the correct code.

The complete finished example could be used in real code, and allow the user to quickly extend their own type to something that can be made to work with all of the `std::simd` APIs, with performant quality code, with relatively minimal effort.

§ 6. Future work

SG6 suggested that all operators should be able to take different input and outputs to facilitate adding `mp-uints`, `constrained_numbers`, and other mixed-type operators. Although this work is incomplete it is still useful to get LEWG feedback on the direction of the other features.

§ 7. Conclusion

In this proposal we have outlined the basic mechanisms needed to allow user-defined types to be stored and manipulated by `std::simd` values, and crucially, to be able to do so without knowledge of the internal implementation of the `std::simd` library.

§ 8. Acknowledgements

We would like to thank Matthias Kretz for his feedback and his useful contributions to discussions.

§ 9. Revision History

R0 => R1

- Incorporated SG1 and SG6 feedback from 2024 Tokyo meeting.
- Added restrictions on element types (e.g., size).
- Added inferencing as a valid method for constructing `simd` operators from scalar operators.
- Added type conversions.
- Removed opt-in and replaced with opt-out.
- Removed explicit user-defined storage.
- Provided inferencing example.

§ References

§ Informative References

[P1928R8]
Matthias Kretz. *std::simd - Merge data-parallel types from the Parallelism TS 2*. 9 November 2023. URL: https://wg21.link/p1928r8

[P2663R4]
Daniel Towner, Ruslan Arutyunyan. *Proposal to support interleaved complex values in std::simd*. 13 October 2023. URL: https://wg21.link/p2663r4